Polytech Clermont

Ingénierie Mathématique et Data Science, 4e année

Segmentation et détection de bulles : une approche par boîte englobante orientée

Amélioration de l'analyse de la distribution de taille des bulles par la détection d'orientation et la segmentation d'instance

Rapport de projet réalisé d'Octobre 2025 à Janvier 2026

Présenté par : Andréa GELIE, Camille RIVIÈRE

Le 11/02/2026

Confidentiel

Tuteur Polytech Clermont : Trong Nghia NGO

Année 2025/2026

0. MOTIVATIONS 3

0.1 PRESENTATION DU SUJET 3

0.2 OBJECTIFS 3

1. ETAT DE L'ART 4

1.1 PREACQUIS 4

1.1.1 APPRENTISSAGE SUPERVISE ET NON SUPERVISE 4

1.1.2 LES RESEAUX DE NEURONES 5

1.1.2.1 Multi-Layer Perceptron (MLP) 5

1.1.2.2 Convolutional Neural Network (CNN) 6

1.2 DETECTION 6

1.2.1 ANCHOR BOXES 6

1.2.2 NON-MAX SUPPRESSION 8

1.2.3 R-CNN, FAST R-CNN ET FASTER R-CNN 8

1.2.4 YOLO (YOU ONLY LOOK ONCE) 9

1.2.5 METRICS 10

1.3 SEGMENTATION 11

1.3.1 MASK R-CNN 11

1.3.2 YOLACT 12

2. EXPERIMENTAL 13

2.1 ANALYSE EXPERIMENTALE DES DONNEES 13

2.2 EXPERIENCES 15

2.3 RESULTATS ET ANALYSES 16

2.3.1 ENTRAINEMENT AVEC YOLO-DET 16

2.3.2 ENTRAINEMENT AVEC YOLO-SEG 20

CONCLUSION 24

BIBLIOGRAPHIE 25

0. MOTIVATIONS

0.1 PRESENTATION DU SUJET

Dans le cadre de l'étude des transferts de masse gaz-liquide, notamment au sein des bioréacteurs, l'analyse du comportement des bulles est un facteur critique. La précision de cette analyse repose sur la mesure de paramètres clés comme le coefficient de transfert de masse volumétrique, l'aire interfaciale par unité de volume, le diamètre moyen de Sauter et l'analyse de la distribution de taille des bulles.

Bien que les méthodes actuelles permettent de détecter les bulles même lorsqu'elles se chevauchent, elles présentent une limite majeure : l'absence de prise en compte de l'orientation de chaque bulle.

0.2 OBJECTIFS

L'objectif principal de ce travail est d'évaluer et de comparer deux approches de vision par ordinateur pour caractériser les bulles dans un bioréacteur. Contrairement à la détection d'objets standard qui ne fournit que des boîtes englobantes droites, nous avons mis en œuvre les solutions suivantes via le framework YOLO :

- **Détection d'objets (YOLO-Det) :** Utilisation de modèles de détection pour localiser chaque bulle, même en cas de chevauchement, afin d'obtenir un comptage précis.
- **Segmentation d'instance (YOLO-Seg) :** Extraction d'un masque pixel par pixel pour chaque bulle détectée. Ce masque permet, par un post-traitement d'ajustement d'ellipse (ellipse fitting), de reconstruire la forme réelle et de déterminer l'orientation précise de la bulle.

Cette double approche permet de corriger les imprécisions des méthodes conventionnelles et d'améliorer la fiabilité des mesures de distribution de taille des bulles.

1. ETAT DE L'ART

1.1 PREACQUIS

1.1.1 Apprentissage supervisé et non supervisé

L'apprentissage supervisé utilise des jeux de données étiquetés pour entrainer des modèles d'IA. Pendant l'entraînement, les labels ou vérités terrains sont utilisés pour guider ces modèles. L'objectif est qu'une fois entraîné, ils soient capables d'inférer correctement à partir de nouvelles données (voir figure 1). L'apprentissage supervisé permet d'effectuer de la régression dans le cas où l'on souhaite prédire une valeur continue ou de la classification dans le cas où ce qui est prédit est une classe.

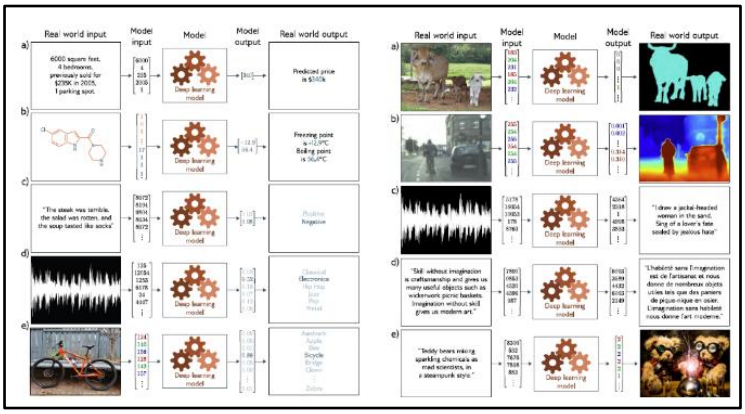


Figure 1 : Exemples d'apprentissages supervisés

L'apprentissage non supervisé, lui, utilise des jeux de données non étiquetés pour ensuite les regrouper en découvrant des modèles cachés dans ceux-ci (voir figure 2). Il permet notamment de réaliser du clustering ou de réduire des dimensions.

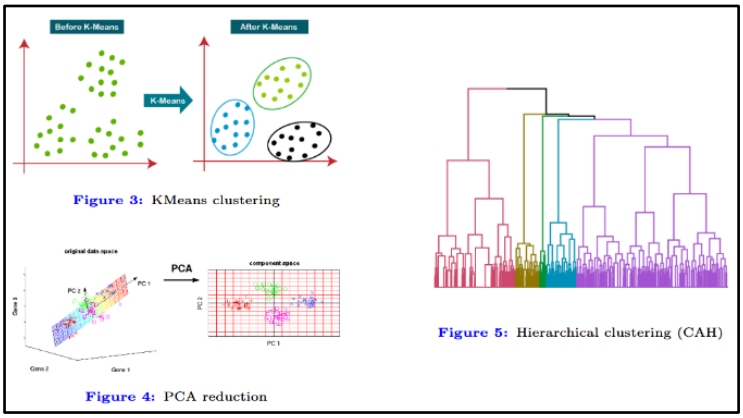


Figure 3: KMeans clustering

Figure 4: PCA reduction

Figure 5: Hierarchical clustering (CAH)

Figure 2 : Exemples d'apprentissages non supervisés

1.1.2 Les Réseaux de Neurones

Le fonctionnement d'un réseau de neurones s'inspire du cerveau humain. Il est composé de neurones artificiels organisés en couches et interconnectés par des poids synaptiques. Chaque neurone reçoit des signaux d'entrée, leur applique une combinaison linéaire pondérée, puis une fonction d'activation non linéaire qui permet au modèle d'apprendre des relations complexes entre les données. Il utilise également la fonction de coût, qui mesure l'erreur entre la prédiction du modèle et la valeur réelle, ainsi que le processus d'optimisation, comme l'algorithme de rétropropagation du gradient (backpropagation), utilisé pour ajuster les poids du réseau afin de minimiser cette erreur.

1.1.2.1 Multi-Layer Perceptron (MLP)

Un MLP est un réseau de neurones composé d'au moins trois couches (voir figure 3) : la couche d'entrée, une ou plusieurs couches cachées et la couche de sortie. Celles-ci ont alors leurs neurones respectifs connectés à ceux de la couche qui les précède. De plus, ces connections entre différentes couches possèdent chacune leur propre poids W auquel on peut ajouter le biais b. Le réseau de neurones prend des vecteurs en entrée et, grâce à ses couches, qui peuvent d'ailleurs posséder un nombre de neurones différent les unes des autres, prédit des valeurs continues qui doivent être proches de la valeur d'une certaine fonction continue passée aux données d'entrée.

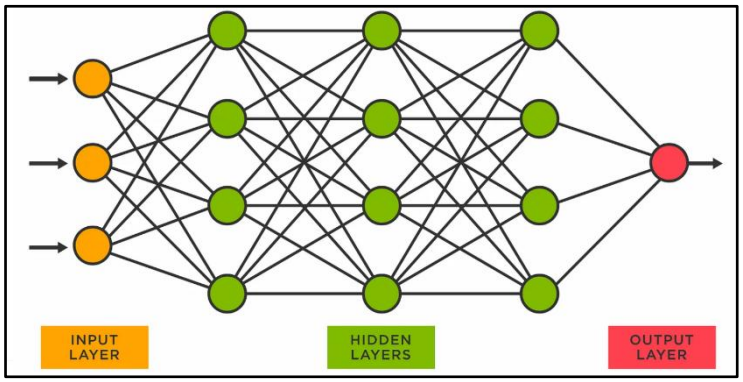


Figure 3 : Exemple de MLP

1.1.2.2 Convolutional Neural Network (CNN)

Le CNN, lui, est bien différent du MLP puisqu'il se compose de couches qui se succèdent et qui ont des fonctions différentes (voir figure 4). De plus, la majeure différence entre ces deux types de réseaux de neurones est le fait que cette fois ci, l'entrée du réseau de neurones est une image. Tout d'abord, on retrouve les couches qui ne traitent qu'une seule partie de l'image en y appliquant des filtres grâce à des opérations de convolution et auxquelles on applique ensuite la fonction Relu. Puis, à la suite de chacune de ces couches, on retrouve une couche de pooling qui permet de réduire la dimensionnalité. Au final, l'utilisation du CNN est elle aussi complètement différente de celle des MLP puisqu'elle consiste à classifier des images et/ou reconnaître des objets.

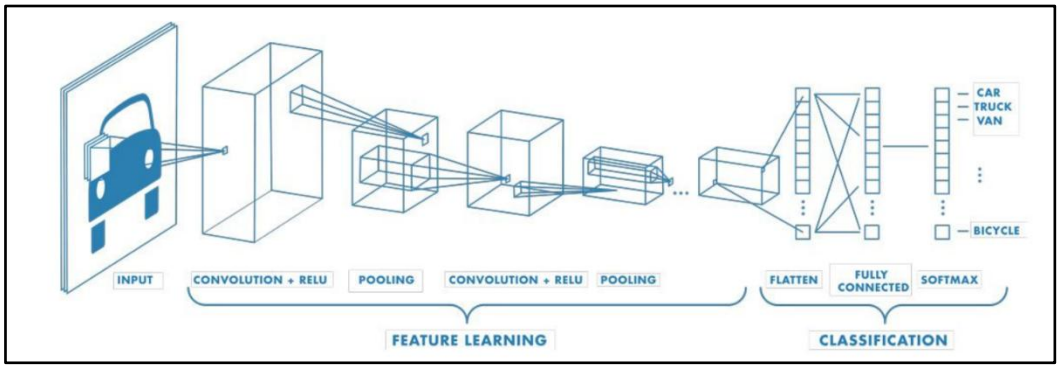


Figure 4 : Exemple de CNN

1.2 DETECTION

1.2.1 Anchor Boxes

Les Anchor Boxes (boîtes d'ancrage)[1] sont un concept clé dans la détection d'objets. Elles permettent au réseau neuronal d'échantillonner efficacement différentes régions d'une image. Ce sont des bounding boxes (contours rectangulaires autour d'un objet ou d'une région d'intérêt dans une image) à échelles variables et centrées sur chaque pixel de l'image et qui permettent notamment d'échantillonner les différentes régions de l'image en entrée du modèle. Le nombre total d'ancres générées est calculé par la formule : *(largeur d'image en input) x (hauteur de l'image en input) x (nombre de combinaisons de scale et aspect ratios).*

Afin d'évaluer la qualité d'une boîte de détection ou le chevauchement entre deux boîtes, on utilise l'Intersection over Union (IoU). Elle est calculée en se basant sur l'index de Jaccard (voir figure 5). C'est une valeur comprise entre 0 (dans le cas où il n'y a pas d'intersection entre les deux boîtes) et 1 (dans le cas où les deux boîtes sont identiques).

$$\text{IoU} = \frac{\text{Aire de l'intersection des deux boîtes}}{\text{Aire de l'union des deux boîtes}}$$

Figure 5 : Expression simplifiée de la formule permettant d'obtenir l'IoU[1]

L'association correcte des anchor boxes aux boîtes réelles (vérité terrain) est une étape primordiale avant l'application de la NMS (Non-Max Suppression). Tout d'abord, l'IoU entre chaque ancre et chaque boîte réelle est calculée puis, les meilleures correspondances sont attribuées. Ensuite, chaque boîte réelle est associée à au moins une ancre (voir figure 6). Les ancres restantes sont ensuite considérées comme positives ou négatives selon un seuil d'IoU. Celles qui sont considérées positives héritent de la classe de l'objet correspondant à ce qu'elles doivent contenir et de valeurs d'offset (le décalage entre l'anchor box et la ground-truth bounding box). Ainsi, chaque ancre devient un exemple d'apprentissage pour le réseau, soit pour prédire un objet, soit pour apprendre à reconnaître le fond.

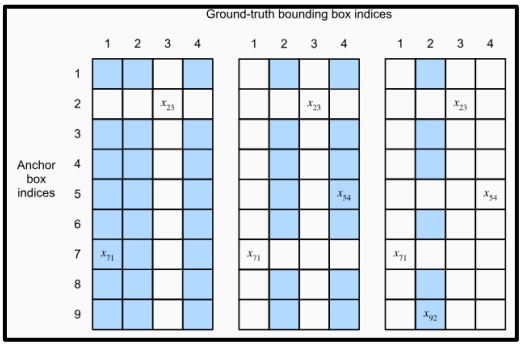


Figure 6 : Association des ground-truth bounding boxes aux anchor boxes[1]

1.2.2 Non-Max Suppression

Lors de la prédiction, après avoir généré plusieurs anchor boxes et prédit leurs classes et offsets, on utilise la NMS (Non-Max Suppression)[1] pour gérer les doublons. La NMS permet de garder la boîte avec le meilleur score de confiance parmi les multiples boîtes qui recouvrent le même objet (voir figure 7).

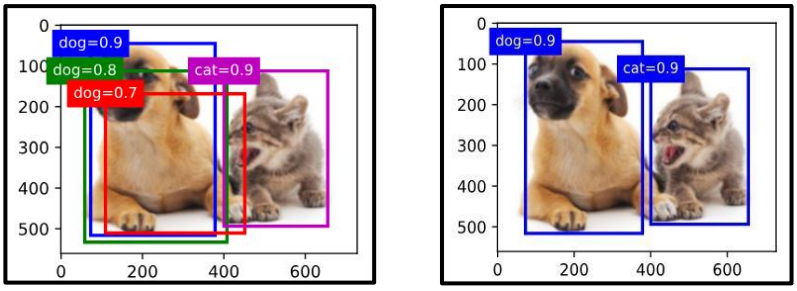


Figure 7 : Comparaison pour une même image avant (à gauche) et après (à droite) l'application du NMS[1]

1.2.3 R-CNN, Fast R-CNN et Faster R-CNN

Avec l'introduction des modèles basés sur les CNN, le système de détection a évolué très rapidement. Ainsi, plusieurs modèles ont été développé et amélioré au fil du temps. On retrouve notamment[2] :

- R-CNN (Region-based CNN)[2] : Ce premier modèle utilise un algorithme externe (Selective Search) pour proposer des milliers de régions d'intérêt, puis applique le CNN sur chacune de ces régions pour la classification et la régression des boîtes. Mais, ce processus est plutôt lent.
- Fast R-CNN[2]: Le Fast R-CNN permet d'accélérer considérablement le processus en effectuant la propagation avant du CNN qu'une seule fois sur l'image entière. De plus, il introduit la couche de RoI (Region of Interest) Pooling pour extraire des caractéristiques de taille fixe à partir des régions d'intérêt, accélérant ainsi l'entraînement et l'exécution. Cependant, le modèle continue d'utiliser l'algorithme externe ce qui le ralentit.

Par la suite, le Faster R-CNN[3] a marqué un tournant en remplaçant la méthode de proposition de régions externe par un réseau neuronal dédié, le Region Proposal Network (RPN). Par conséquent, le système de détection d'objet Faster R-CNN se compose de deux modules qui forment un unique réseau. Le premier est un réseau convolutionnel profond chargé de renvoyer les zones candidates pouvant potentiellement contenir un objet tandis que le second est un détecteur Fast R-CNN qui utilise les régions définies par le premier module. La principale

innovation de ce modèle consiste à utiliser un RPN (Region Proposal Network) qui permet ainsi au modèle de générer des propositions de régions de haute qualité.

$$L = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$$

Figure 8 : Fonction de perte du modèle Faster R-CNN[3]

1.2.4 YOLO (You Only Look Once)

YOLO (*You Only Look Once*)[4] est une approche véritablement innovante pour la détection d'objets. En effet, contrairement aux méthodes utilisées auparavant, qui réutilisaient des classificateurs pour détecter les objets, nous avons noté que YOLO considère la détection comme un problème de régression vers des boîtes englobantes et les probabilités de classes associées. Ainsi, un seul réseau de neurones prédit directement les boîtes et les classes à partir de l'image entière, ce qui permet d'optimiser le réseau de bout en bout pour la performance de détection.

La rapidité est l'un des atouts majeurs de YOLO. Effectivement, le modèle standard traite les images à une vitesse de 45 images par seconde, et la version Fast YOLO peut même atteindre les 155 images par seconde. De plus, YOLO apprend des représentations générales des objets, ce qui lui permet de se généraliser efficacement à de nouveaux domaines, y compris les œuvres d'art, ce qui lui permet ainsi de surpasser d'autres méthodes comme DPM[4] ou R-CNN.

Pour parvenir à unifier tous les composants de la détection d'objets en un seul réseau de neurones, YOLO utilise les caractéristiques de l'image entière pour prédire simultanément les boîtes englobantes et les probabilités de classes, ce qui lui donne la capacité de raisonner globalement sur tous les objets présents dans l'image. Cette approche simplifie grandement le pipeline de détection et permet un entraînement de bout en bout, tout en garantissant des vitesses de traitement en temps réel et une précision élevée.

Regression loss

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$
$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

Confidence loss

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2$$
$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2$$

Classification loss

$$+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

Figure 9 : Fonction de perte de YOLO[4]

Plus précisément, Le modèle fonctionne en divisant l'image d'entrée en une grille de dimension S*S. Si le centre d'un objet se situe dans une cellule, cette cellule est alors responsable de la détection de cet objet. Ensuite, chaque cellule prédit B boîtes englobantes

avec leurs scores de confiance, ainsi que C probabilités conditionnelles de classes. Finalement, les scores de confiance combinent la probabilité qu'un objet soit présent et l'exactitude de la boîte prédite ; ces probabilités conditionnelles sont multipliées par les scores de confiance pour obtenir les scores finaux de chaque classe.

Enfin, nous devons toutefois souligner qu'il existe des marges d'amélioration. Bien que YOLO présente moins de faux positifs que d'autres méthodes pour définir l'arrière-plan d'une image, nous avons noté qu'il fait davantage d'erreurs de localisation, notamment lorsque les objets à détecter sont particulièrement petits.

1.2.5 Metrics

L'évaluation des modèles de détection repose sur des métriques clés qui doivent combiner à la fois la précision de la localisation et la justesse de la classification.

On retrouve tout d'abord l'Intersection sur Union ou IoU qui mesure notamment le chevauchement entre la boîte englobante prédite (Bp) et la boîte de vérité terrain (Bgt) (voir figure 10).

$$\text{IoU} = \frac{\text{Aire}(B_p \cap B_{gt})}{\text{Aire}(B_p \cup B_{gt})}$$

Figure 10 : Formule mathématique permettant de calculer l'IoU[5]

La Précision, elle, mesure la proportion de prédictions positives correctes parmi toutes les détections positives effectuées par le modèle (voir figure 11).

$$\text{Précision} = \frac{TP}{TP + FP}$$

Figure 11 : Formule permettant de calculer la Précision[5]

Tandis que le Rappel, lui, mesure la proportion de toutes les instances positives qui ont été correctement détectées (voir figure 12).

$$\text{Rappel} = \frac{TP}{TP + FN}$$

Figure 12 : Formule permettant de calculer le Rappel[5]

Enfin, il y a la Mean Average Precision ou mAP qui est la métrique utilisée principalement pour évaluer les performances des modèles de détections d'objets. Elle peut permettre à la fois de mesurer la précision de détection et la qualité de la classification des objets détectés. Pour la calculer, on utilise l'AP (Average Precision) qui permet de mesurer la

performance d'un modèle qui ne détecte qu'une seule classe et dont la valeur correspond à celle de l'aire sous la courbe Précision-Rappel.

Ainsi, lorsqu'on calcule la mAP pour une classe unique, on la définit par la formule suivante (voir figure 13) :

$$\text{mAP} = \frac{1}{10}\frac{1}{N}\sum_{t=0}^{9}\sum_{k=0}^{N-1}\text{AP}_{r_k}^{\tau_t}, \qquad r_k = \frac{k}{N-1}, \quad \tau_t = 0.5 + t \times 0.05.$$

Figure 13 : Calcul permettant d'obtenir la mAP pour une classe unique

Et, si l'on souhaite calculer la mAP pour un modèle qui possède plusieurs classes, on reproduit la méthode précédente pour chacune des classes du modèle avant dans faire la moyenne (voir figure 14).

$$\text{mAP} = \frac{1}{C}\sum_{i=0}^{C-1} mAP_i,$$

Figure 14 : Calcul de la moyenne des mAP de chaque classe du modèle

1.3 SEGMENTATION

1.3.1 Mask R-CNN

Mask R-CNN[6] est une extension du Faster R-CNN qui reprend la même structure de base, mais y ajoute une troisième branche chargée de prédire un masque de segmentation pour chaque région d'intérêt (RoI) (voir figure 15). Cette branche prend la forme d'un petit réseau entièrement convolutionnel (FCN). Le modèle peut également être adapté à d'autres tâches, comme l'estimation de la pose d'un humain en étudiant ses points clés.

Pour obtenir des masques précis, Mask R-CNN doit préserver la structure spatiale fine des objets. C'est pourquoi il introduit la couche RoIAlign, qui supprime la quantification présente dans RoIPool et permet un alignement exact entre les régions extraites et les pixels d'origine. Cette étape est essentielle pour la prédiction de masques pixel-par-pixel, et constitue l'une des principales améliorations par rapport au Faster R-CNN, améliorant nettement la qualité des segmentations. Ainsi, grâce à cette modification, Mask R-CNN devient un modèle polyvalent

pour la segmentation d'instances, qui combine classification, localisation des objets et prédiction de masques précis.

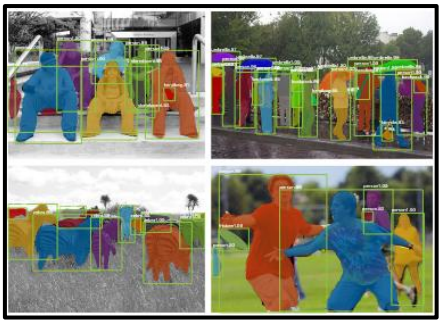


Figure 15 : Exemples de résultats de Mask R-CNN sur des images test de COCO[6]

1.3.2 YOLACT

YOLACT[7] un modèle simple, entièrement convolutionnel, utilisé pour la segmentation d'instances en temps réel, qui est nettement plus rapide que toutes les approches précédentes et qui s'entraine sur un seul GPU.

Il fonctionne notamment en divisant la segmentation d'instances en deux sous-tâches parallèles. Tout d'abord, il génère un ensemble de masques prototypes, et ensuite, il prédit des coefficients de masque par instance (voir figure 16). De plus, comme ce procédé ne dépend pas d'un rééchantillonnage, il permet d'obtenir des masques de très haute qualité et offre gratuitement une stabilité temporelle.

En effet, la division en sous tâches aura notamment permis à YOLACT d'être reconnue comme étant la première méthode de segmentation d'instances très rapide là où les autres ne se focalisaient pas sur la vitesse. Elle atteint facilement les 30 images par seconde. Mais, cette approche à d'autres avantages comme le fait que les masques renvoyés soient de très bonne qualité et qu'elle soit générale.



Figure 16 : Résultats de l'évaluation de YOLACT sur l'ensemble test-dev de COCO[7]

2. EXPERIMENTAL

2.1 ANALYSE EXPERIMENTALE DES DONNEES

Dans le dataset utilisé, les images étaient de taille 512 x 512 pixels. Celui-ci est séparé en deux parties, yolo_seg et yolo_det, dans lesquels on retrouve deux dossiers : le dossier train qui est composé de 98 images et le dossier val composé de 10 images. Ces images sont les mêmes pour deux tâches (segmentation et détection), la différence réside dans l'organisation des labels. Pour la tâche de détection, chaque fichier .txt contient une annotation par ligne d'un objet (classe xc yc h w), où xc, yc (centre d'un objet), h et w (hauteur et largeur d'un objet) sont normalisés entre 0 et 1. Un même fichier .txt peut inclure plusieurs annotations. En revanche, l'annotation de segmentation présente un polygone défini par N points (classe x1 y1 … xN yN), tous les xi, yi i allant de 1 à N étant également normalisés entre 0 et 1.

De même pour les objets, ils sont à peu près répartis de manière équivalente dans les deux datasets. La répartition est la suivante :

- yolo_det : train : 2360 objets / val : 261 objets
- yolo_seg : train : 2361 objets / val : 261 objets

Au sein du dataset, on retrouve 4 classes différentes qui sont intitulées "back","bord","fore" et "full" et qui désignent notamment la disposition des bulles au sein des images. "Back" indique des bulles cachées derrière une ou plusieurs autres bulles ; "bord" désigne les bulles situées aux bordures des images ; "fore" représente les bulles devant une ou plusieurs autres bulles ; et "full" décrit des bulles complètes et isolées. L'existence de ces quatre classes est reconnue mais nous avons remarqué la présence d'une cinquième classe dont nous ne connaissons pas l'intitulé lorsque nous avons étudié les labels présents dans le dossier train des datasets. Par exemple, on la retrouve dans « gaz_150_eau_1200_000157_71 » dans le dossier train de yolo_det.

A l'intérieur des différents splits, les classes sont réparties de la manière suivante (voir figure 17) :

Classe		0	1	2	3	4
yolo_det	**train**	310	507	257	1283	3
	val	25	37	24	175	0
	TOTAL	**335**	**544**	**281**	**1458**	**3**
yolo_seg	**train**	310	507	258	1283	3
	val	25	37	24	175	0
	TOTAL	**335**	**544**	**282**	**1458**	**3**

Figure 17 : Tableau illustrant la répartition des classes aux sein des splits

Ce tableau ainsi que les histogrammes suivants (voir figures 18, 19 et 20) illustrent bien le fait que tout comme les objets, les classes sont réparties de façon similaires dans les différents splits. De plus, nous avons notamment remarqué que ces histogrammes confirment l'existence de la cinquième classe inconnue parmi certaines images destinées à l'entraînement des modèles. Enfin, la classe 3 qui est la classe intitulée « full » est la classe la plus représentée au sein du dataset.

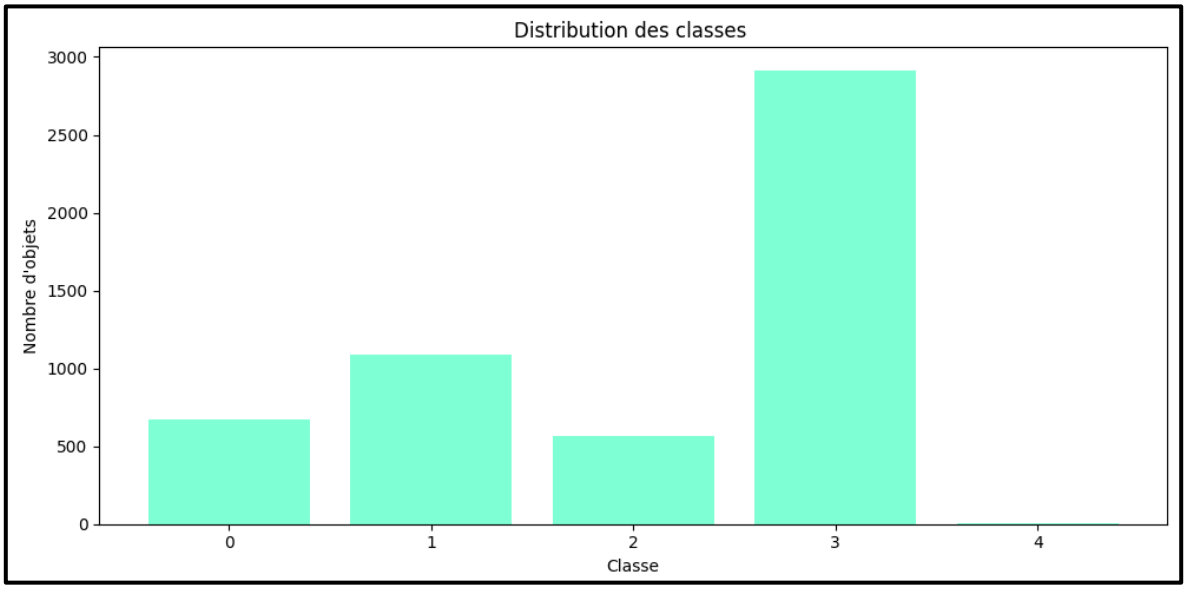


Figure 18 : Histogramme illustrant la distribution des classes dans l'entièreté du jeu de données

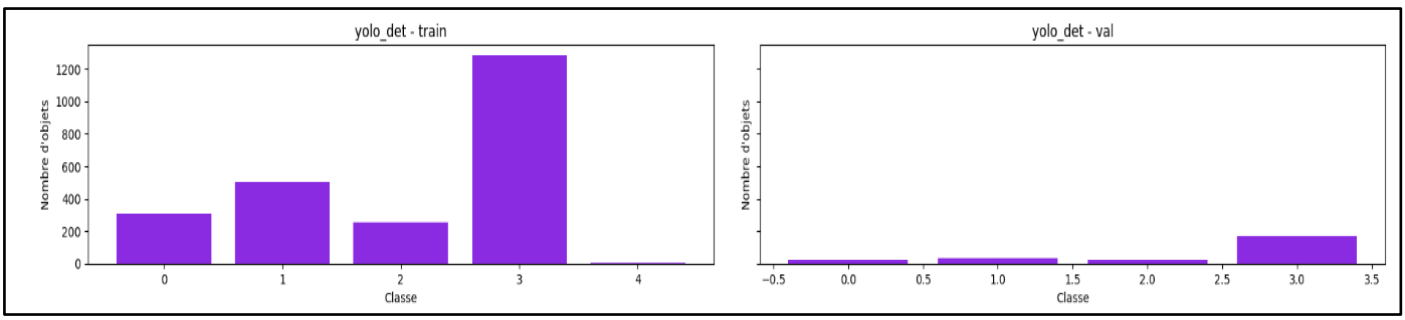


Figure 19 : Histogramme illustrant la distribution des classes dans le dataset yolo_det

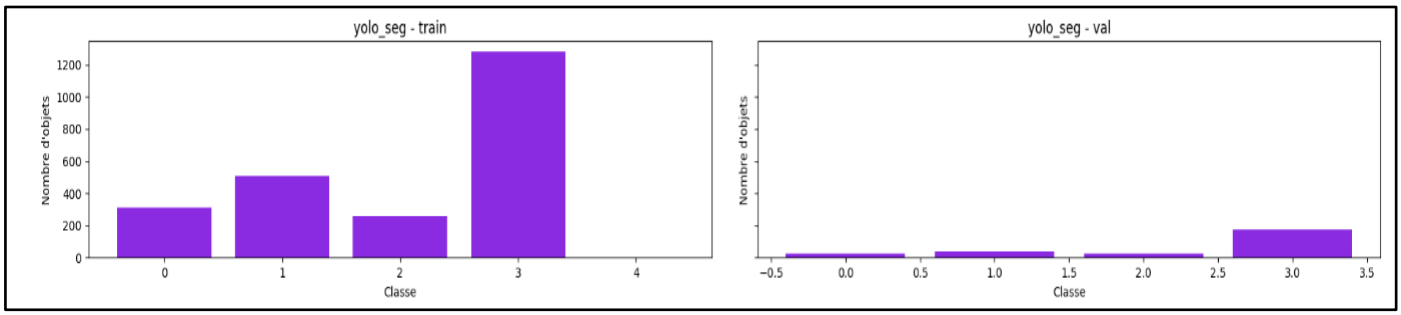
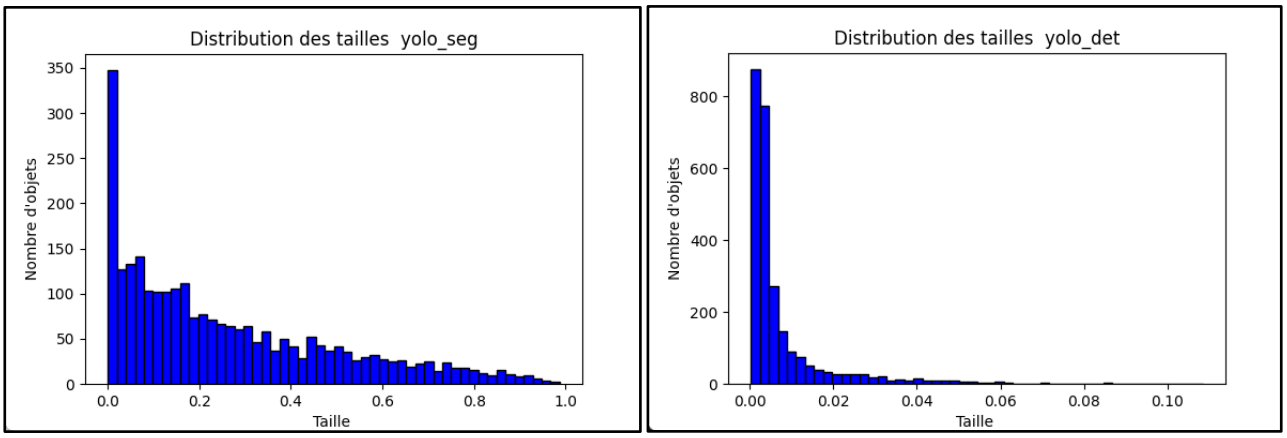


Figure 20 : Histogramme illustrant la distribution des classes dans le dataset yolo_seg

De plus, les différentes tailles d'objets se distribuent de manière similaire dans les deux jeux de données (voir figures 21 et 22). On donc que la majorité des objets à identifier au sein des images, qui sont donc ici des bulles, sont majoritairement de taille extrêmement faible, ce qui, comme nous l'avons expliqué auparavant, peut poser des difficultés à YOLO lors de la détection d'objets.



Figures 21 et 22 : Histogrammes illustrant la distribution des tailles sur l'aire h*w des objets dans yolo_seg (à gauche) et yolo_det (à droite)

2.2 EXPERIENCES

L'entraînement d'un modèle YOLO[8], souvent réalisé avec des frameworks comme Ultralytics, vise à optimiser la convergence et la performance.

Il existe notamment un certain nombre d'hyperparamètres cruciaux pour le fonctionnement de YOLO tels que :

- Le nombre d'époques (itérations sur l'ensemble des données)
- La taille du lot (batch size), qui définit le nombre d'images traitées simultanément.
- L'algorithme d'optimisation (par exemple SGD ou Adam) qui est guidé par le taux d'apprentissage initial, qui est lui-même souvent ajusté dynamiquement au fil des époques.

Dans le cadre de notre projet, nous avons utilisé Ultralytics afin de pouvoir entrainer les modèles Yolo11n et Yolo11n-seg respectivement sur les datasets yolo_det et yolo_seg.

De plus nous avons ajusté différents hyperparamètres parmi ceux les plus cruciaux pour le fonctionnement du modèle afin d'obtenir les meilleurs résultats possibles, et cela assez facilement via YAML. Nous avons donc, au final, entrainé les modèles mentionnés dans le paragraphe précédent avec une taille d'images de 512 pixels afin de respecter la taille réelle de celles passées en entrée des modèles et nous avons choisi de faire 40 époques pour Yolo11n et 55 pour Yolo11n-seg pour assurer une métrique finale.

2.3 RESULTATS ET ANALYSES

2.3.1 Entraînement avec YOLO-Det

Nos résultats en sortie du modèle semblent être assez prometteurs puisque le modèle parvient à détecter l'ensemble des objets présents sur les images. En revanche, le modèle ne semble pas tout le temps associer les bonnes classes aux objets correspondant (voir figure 23).

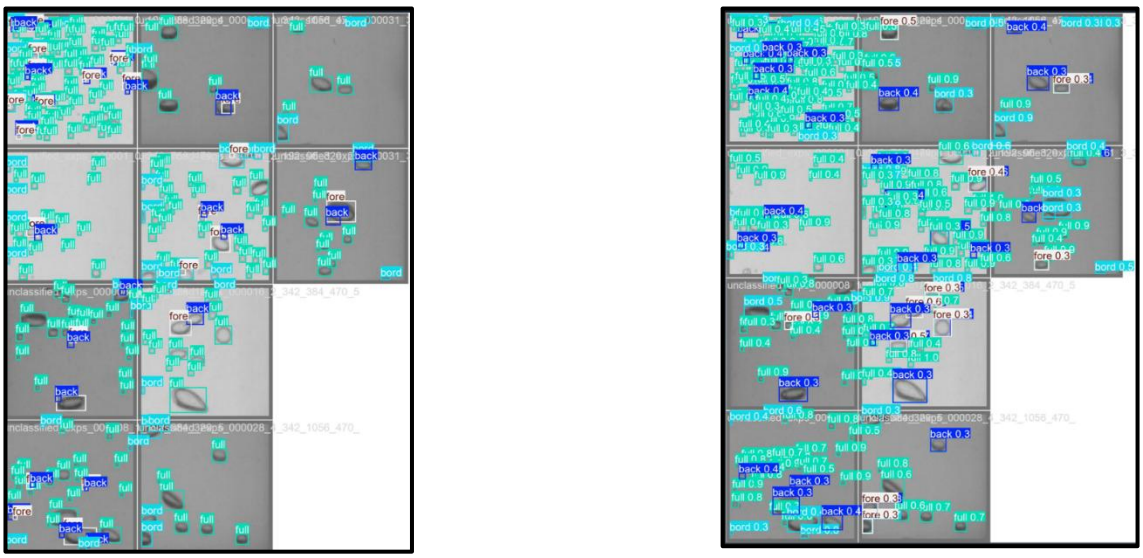


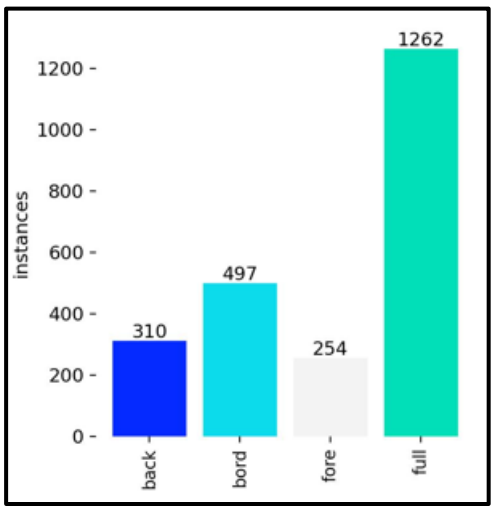
Figure 23 : Comparaison entre les labels (à gauche) et les prédictions du modèle (à droite)

En regardant de plus près les courbes Precision-Recall pour chacune des classes (voir figure 25), ces différences sont d'autant plus flagrantes. En effet, la classe « full » est la classe la plus performante avec un mAP50 de 0,921. Nous l'avons massivement alimentée avec 1 283 instances en entraînement, ce qui explique pourquoi notre modèle la maîtrise si bien.

Pour la classe « bord », nous obtenons un score correct de 0,758 (mAP50). Avec nos 507 exemples en phase de train, nous avons donné au modèle assez de matière pour identifier correctement cette catégorie.

Pour la classe "back", nous tombons à un mAP50 très faible de 0.134, et, pour la classe "fore", nous sommes en situation d'échec quasi total avec un mAP50 de 0.085, et nous voyons même des scores de précision et de rappel tomber à 0. Lors de l'entraînement, le modèle a rencontré respectivement 310 et 257 instances de celles-ci.

Cette hétérogénéité des scores semble directement liée à la manière dont nos classes sont réparties dans notre dataset (voir figure 24).



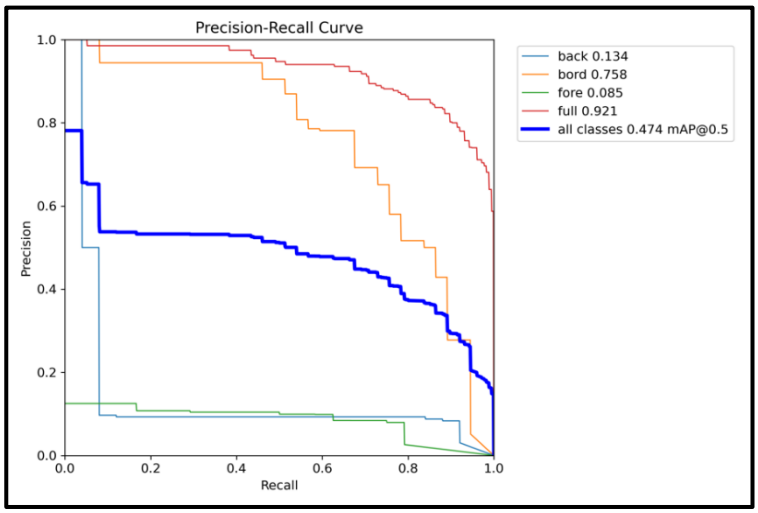


Figure 24 (à gauche) : Histogramme illustrant la distribution des instances d'objets effectivement exploitées par le modèle.

Figure 25 (à droite) : Courbe Precision-Recall pour chacune des classes présentes dans le jeu de données.

De plus, en observant les valeurs qui se trouvent sur la diagonale de la matrice de confusion (voir figure 26), ces mêmes observations sont effectivement confirmées. On retrouve respectivement des scores de 0.88 et 0.51 pour les classes « full » et « bord » tandis que les deux autres classes n'ont même pas de score. La classe « fore » est trop fréquemment prédite comme « full » ou le fond, avec des taux de confusion de 0.58 et 0.33 respectivement. Cette situation se manifeste également pour la classe « back », présentant des taux d'erreur de 0.72

et 0.2 respectivement. Une autre observation est la prédiction dominante des classes « full ». Pour expliquer ce dernier phénomène, il s'agit vraisemblablement de la prédominance de cette classe au sein des jeux de données étudiés ; les modèles seraient donc biaisés par la classe « full ».

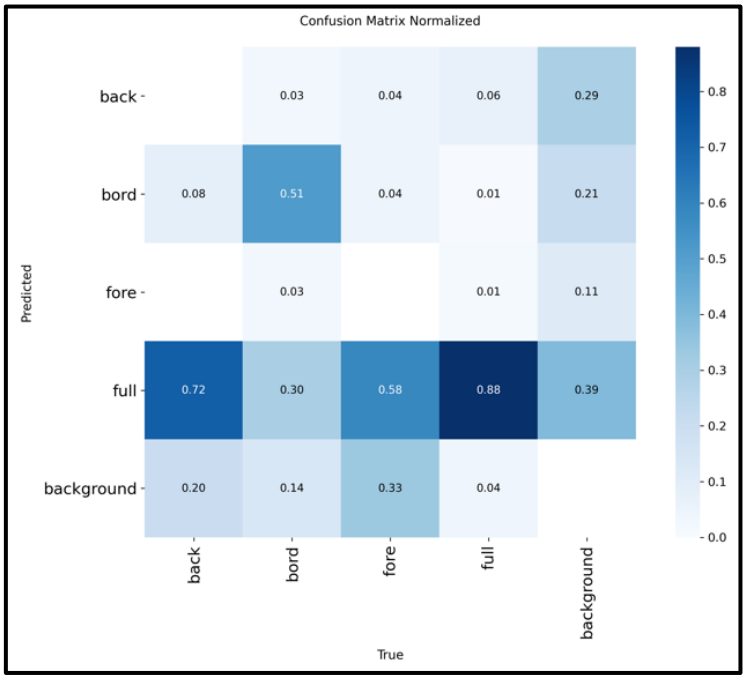


Figure 26 : Matrice de confusion en sortie de l'entraînement avec YOLO-Det

Le graphique de distribution des tailles des objets (voir figure 27), quant à lui, nous donne une explication supplémentaire concernant nos difficultés actuelles. En effet, la grande majorité de nos labels sont regroupés dans le coin inférieur gauche, avec des dimensions souvent variées autour 0.05*512=25.6 pixels, ce qui signifie que nous travaillons sur de l'infiniment petit. Comme nous le savons, les petits objets sont les plus complexes à détecter. De ce fait, il est bien plus difficile pour notre modèle de prédire si les bulles sont plutôt situées à l'arrière-plan ou au premier plan.

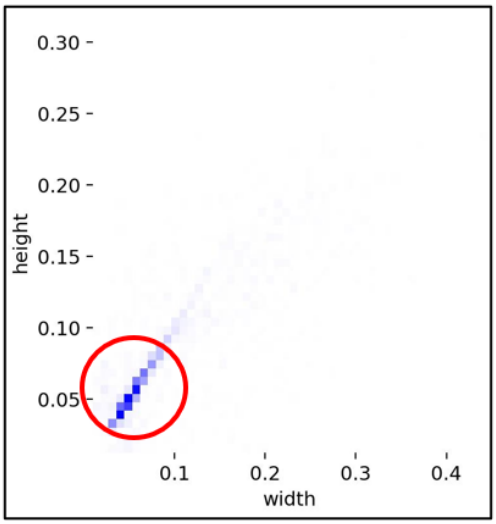


Figure 27 : Graphique montrant la distribution des tailles des objets exploités par le modèle

Enfin, nous avons pu constater que le nombre d'époques que nous avons choisi pour l'entraînement de notre modèle, qui est de 40, semble être pertinent puisqu'en observant les courbes du mAP, nous avons pu constater qu'elles commençaient à stagner (voir figure 28).

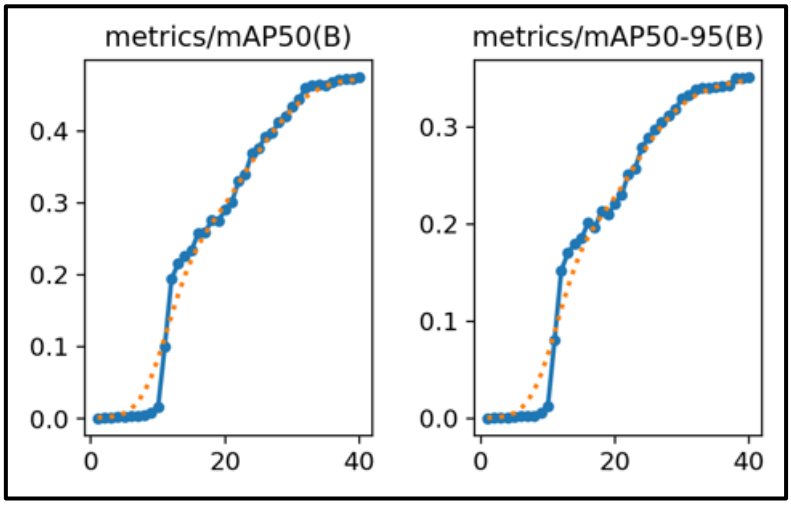


Figure 28 : Courbes représentant le score mAP du modèle après chaque époque

2.3.2 Entraînement avec YOLO-Seg

Nos résultats en sortie du modèle YOLO-Seg semblent être eux aussi assez bon et prometteurs puisque le modèle parvient à détecter l'ensemble des objets présents sur les images et génère également des masques qui semblent bien recouvrir les silhouettes des objets. En revanche, le modèle a encore une fois eu des problèmes lors de l'attribution des classes aux différents objets (voir figure 29).

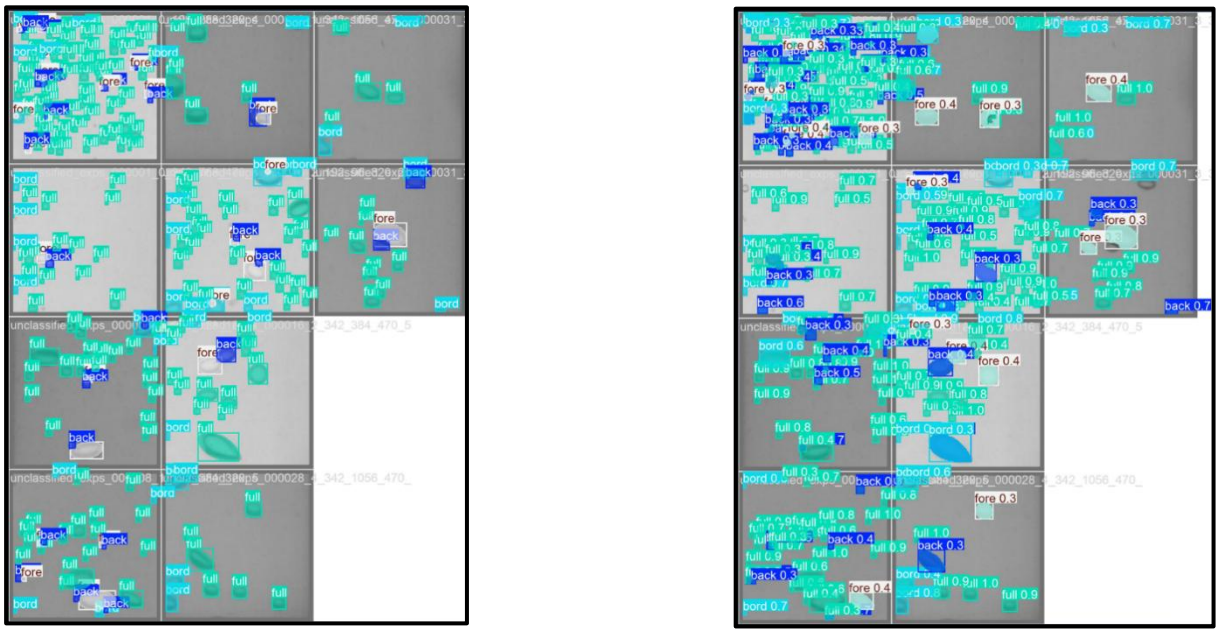
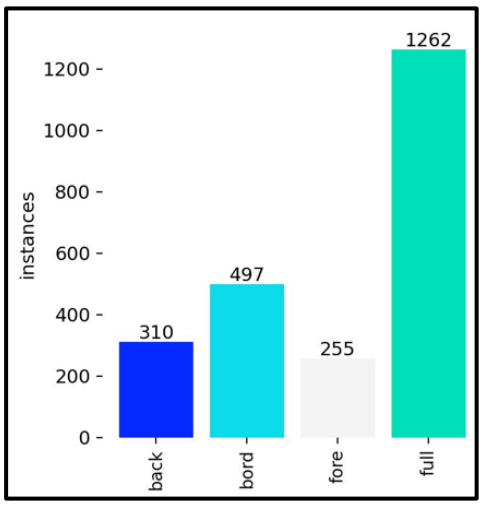


Figure 29 : Comparaison entre les labels (à gauche) et les prédictions du modèle (à droite)

En observant plus en détails les métriques que nous avons obtenu pour YOLO-Seg, nous remarquons que les performances restent assez similaires à celle de YOLO-Det, mais avec tout de même quelques nuances (voir figure 31).

La classe « full » est toujours aussi bien identifiée avec un mAP50 (Mask) de 0,927. De même pour la classe « bord », avec laquelle nous atteignons un mAP50 de 0,815 ce qui est même une amélioration par rapport au modèle précédent. Cela montre donc que notre modèle parvient à détourer les contours des bulles sur les bords de manière efficace. Pour la classe "back", nous obtenons un mAP50 de 0,256. C'est une progression par rapport à YOLO-Det, mais cela reste insuffisant pour une exploitation fiable. En revanche, "fore" reste le point faible de notre modèle avec un mAP50 de 0,177.



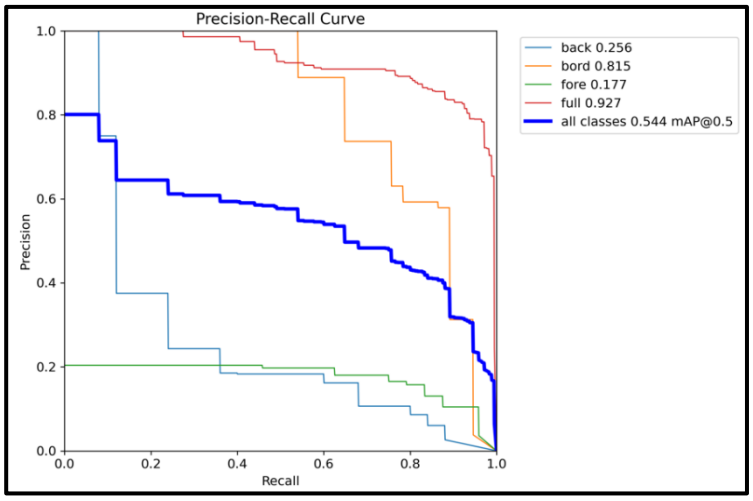


Figure 30 (à gauche) : Histogramme illustrant la distribution des instances d'objets effectivement exploitées par le modèle.

Figure 31 (à droite) : Courbe Precision-Recall pour chacune des classes présentes dans le jeu de données.

En observant les valeurs qui se trouvent sur la diagonale de la matrice de confusion (voir figure 32), on retrouve respectivement des scores de 0.83 et 0.68 pour les classes « full » et « bord ». Cela nous montre donc qu'une amélioration a eu lieu entre les deux modèles concernant la classe « bord ». De plus, cette fois-ci, des valeurs, bien que très faibles, sont présentes sur la diagonale de la matrice pour les classes « back » et « fore ». Pour ces deux classes, la valeur est de 0.04. En effet, la détection ne prend en compte que la position et la taille des objets, tandis que la segmentation inclut les caractéristiques intérieures définissant les polygones, ce qui explique l'amélioration de la prédiction de classe par rapport aux résultats obtenus avec la détection.

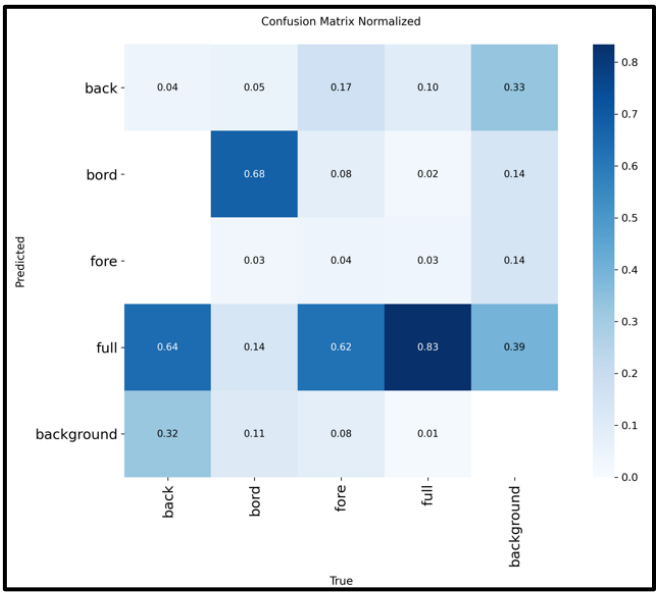


Figure 32 : Matrice de confusion en sortie de l'entraînement avec YOLO-Seg

Le graphique de distribution des tailles des objets (voir figure 33), quant à lui, nous montre qu'ici il est encore question d'objets de taille très faible. En effet, la grande majorité de nos labels sont encore une fois regroupés dans le coin inférieur gauche, avec des dimensions souvent inférieures à 0,1. De ce fait, notre modèle à encore connu des difficultés lors de la prédiction des classes.

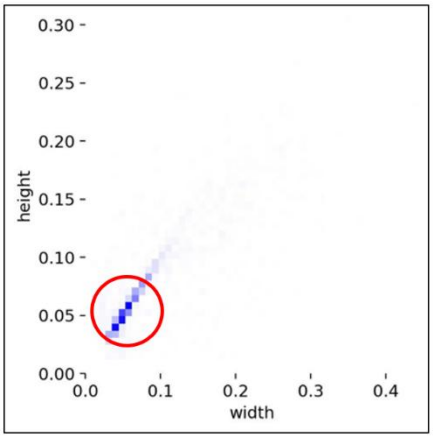


Figure 33 : Graphique montrant la distribution des tailles des objets exploités par le modèle

Concernant la mAP, celle-ci semble tout juste commencer à stagner (voir figure 34). Cela semble donc signifier que le nombre d'époques que nous avons choisi pour l'entraînement de notre modèle, qui est de 55, est pertinent.

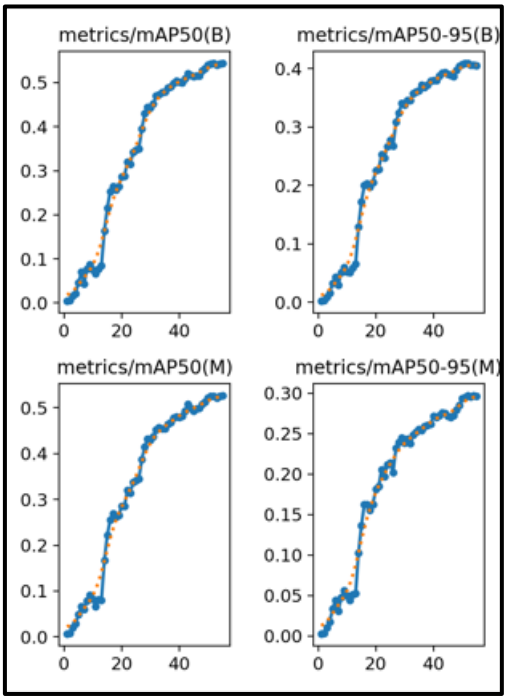


Figure 34 : Courbes représentant le score mAP du modèle après chaque époque

Finalement, ces résultats nous montrent que le modèle de segmentation semble tout de même être le plus performant des deux. Cette impression est notamment renforcée lorsque l'on compare la courbe du mAP, correspondant à la moyenne du mAP de chacune des classes, des deux modèles (voir figure 35).

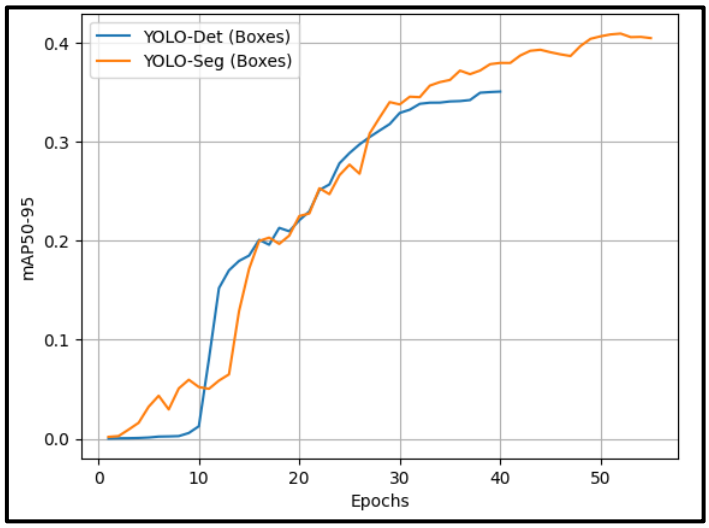


Figure 35 : Comparaison entre le mAP de YOLO-Det et de YOLO-Seg

CONCLUSION

Ce projet, mené d'octobre 2025 à janvier 2026, nous a permis de mettre en œuvre les modèles de Deep Learning YOLO-Det et YOLO-Seg pour l'analyse des bulles en milieu diphasique. L'objectif principal était de dépasser les limites de la détection conventionnelle pour améliorer la précision des mesures de distribution de taille.

Nos objectifs de localisation ont été atteints puisque les modèles détectent efficacement les bulles, même en cas de chevauchement. Cependant, une analyse des performances montre que si la détection est robuste, la classification entre les catégories, elle, reste perfectible. Nous avons observé que les modèles classifient parfois mal les objets en raison de la dominance de la classe « full ». De plus, la forte similarité visuelle entre cette même classe et la classe « fore » génère une confusion pour la machine comme pour l'humain. Pour optimiser les métriques, une solution pertinente serait par exemple de fusionner ces catégories.

Sur le plan pédagogique, ce projet a constitué une expérience d'apprentissage enrichissante dans le domaine de la vision par ordinateur appliquée à la recherche scientifique puisque nous avons pu effectuer une mise en pratique concrète des modèles de Deep Learning via le framework YOLO. En plus de cela, nous avons appris à utiliser de la littérature scientifique de manière pertinente lors de la réalisation d’un projet et également à aborder une problématique scientifique de manière autonome.

BIBLIOGRAPHIE

[1] « 14.4. Anchor Boxes — Dive into Deep Learning 1.0.3 documentation ». Consulté le: 31 janvier 2026. [En ligne]. Disponible sur: https://d2l.ai/chapter_computer-vision/anchor.html

[2] « 14.8. Region-based CNNs (R-CNNs) — Dive into Deep Learning 1.0.3 documentation ». Consulté le: 31 janvier 2026. [En ligne]. Disponible sur: https://d2l.ai/chapter_computer-vision/rcnn.html#r-cnns

[3] S. Ren, K. He, R. Girshick, et J. Sun, « Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks », 6 janvier 2016, *arXiv*: arXiv:1506.01497. doi: 10.48550/arXiv.1506.01497.

[4] J. Redmon, S. Divvala, R. Girshick, et A. Farhadi, « You Only Look Once: Unified, Real-Time Object Detection », 9 mai 2016, *arXiv*: arXiv:1506.02640. doi: 10.48550/arXiv.1506.02640.

[5] « Evaluating Object Detection Models: Methods and Metrics », GeeksforGeeks. Consulté le: 31 janvier 2026. [En ligne]. Disponible sur: https://www.geeksforgeeks.org/computer-vision/evaluating-object-detection-models-methods-and-metrics/

[6] K. He, G. Gkioxari, P. Dollár, et R. Girshick, « Mask R-CNN », 24 janvier 2018, *arXiv*: arXiv:1703.06870. doi: 10.48550/arXiv.1703.06870.

[7] D. Bolya, C. Zhou, F. Xiao, et Y. J. Lee, « YOLACT: Real-time Instance Segmentation », 24 octobre 2019, *arXiv*: arXiv:1904.02689. doi: 10.48550/arXiv.1904.02689.

[8] « Entraînement de modèles avec Ultralytics YOLO - Docs Ultralytics YOLO ». Consulté le: 31 janvier 2026. [En ligne]. Disponible sur: https://docs.ultralytics.com/fr/modes/train/#usage-examples